

LECTURE :3

Computer Bus

Q.9 What is a computer Bus? Explain its basic types.

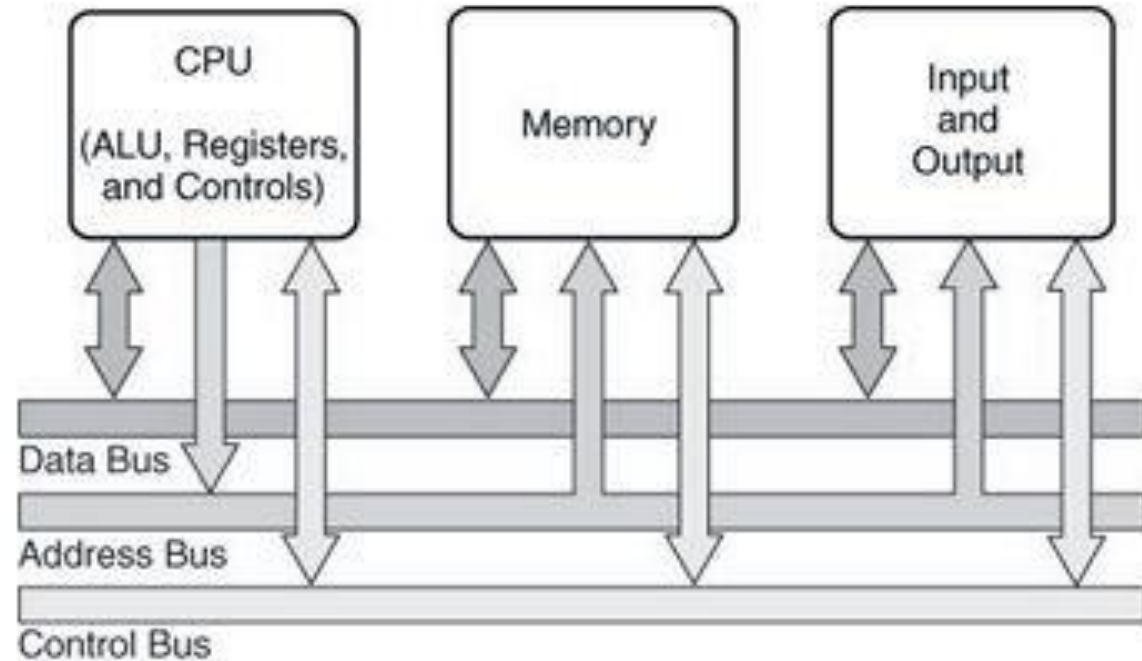
Computer Bus

- Different components of computer interconnected by using a set of parallel lines or conducting wires
- Computer Bus set of electrical paths through which computer (CPU) sends & receives data & instructions (and also sends command signals) to/from different components of computer
- Backbone of computer
- Without bus Computer cannot perform any function • Various devices or components communicate through buses
- Example: buses transfer data
- input device memory
- memory output device or storage device or between memory and processor
- Capacity of bus depends upon number of data lines
- For example: bus with 16-lines carry 16-bits at a time
- **Bus Width** amount of data (or a number of bits) that a Bus can carry at one time • For example: bus with 32 lines transfer 32 bits (or 4 bytes) at a time
- Larger number of bits handled by bus faster data transfer rate

Types of Buses

(1) System Buses

- Connects main components of computer
- such as CPU & main memory on motherboard
- System Buses part of motherboard
- Normally 70 – 100 lines
- Types of System Bus
- Data Bus
- Address Bus
- Control Bus



(2) Expansion Buses

- Connect CPU (processor) to peripheral devices
- Peripheral devices: keyboard, mouse, modem, printer, etc.
- Processor communicates with peripheral via expansion bus & system bus
- Width of expansion buses from 16 to 64 bits

Types of System Bus

1. Data Bus

- Transfer data (and instructions) between different components
- Connects CPU, memory unit, and other hardware devices on motherboard • *bi-directional*
 - Data sent & received through this electrical path
 - For example: CPU write & read data to/ from main memory through a *data bus*
- Consists of 8, 16, 32, or 64 parallel lines
 - Data Bus of 64-line transfer 64-bits (or 8 bytes) of data at a time
- Number of lines affects speed of data transfer

2. Address Bus

- Carry address information or memory locations
- *Unidirectional*
- information flows only in one direction
- CPU can only send address information & cannot receive any information
- Components connected through *address bus*
- Each component assigned unique ID *address* of that component
- When a computer component needs to communicate
- uses address bus to specify address of that component
- Suppose CPU needs to read data from memory
- places address of that memory location on *address bus*
- address is carried to memory

Types of System Bus

- Data from requested memory location fetched
- placed on *data bus*
- CPU reads data from *data bus*
- Width of *address bus* from 16 to 32 bits
- 32-bit address bus address 2^{32} (4,294,967,296 i.e. 4GB) memory locations

3. Control Bus

- CPU sends control signals to different components or devices
- Control signal *timing information & command signal*
 - **Timing information** time for which device or component can use *data & address bus*
 - **Command signal** specifies the type of operation that is to be performed
- CPU (control unit) controls functioning of components through *control bus*
 - Components & devices input/output devices, main memory, secondary storage devices, etc.
- Width of control bus from 8 to 16 bits
- Suppose CPU wants to read data from main memory
 - *Control bus* send "memory read" command to main memory
- Also transmit other control signals like ACKS (Acknowledgement signals)
 - When CPU sends command to main memory writing data
 - memory sends back acknowledgment signal to CPU after writing data successfully

CPU Commands CPU operations

•Memory Write

- Used to write data to a specified location in main memory

•Memory Read

- Used to read data from a specified location in main memory

•I/O Write

- Used to write data to an output device

•I/O Read

- Used to read data from an input device

•Bus Request

- Used to request for taking control of Bus
- so that requesting device can use it to transmit data

•Bus Grant

- Used by Bus controller to indicate grant of the Bus to a device

Data Communication: CPU Memory

- Suppose CPU wants to read data from main memory
 - Control bus "memory read" command send to main memory
 - Address bus CPU places address of memory location
 - Address carried to memory
 - Data from requested memory location fetched
 - placed on *data bus*
 - CPU reads data from *data bus*

If Number of components connected to system bus increase, then

- More components try to use System Bus at same time
 - Slow down computer
 - Because; components have to wait longer to get access to Bus
 - Only major components are connected
 - Remaining components are connected to another external Bus Expansion Bus
- Solution of Problem:
- Expansion Bus connected to System Bus

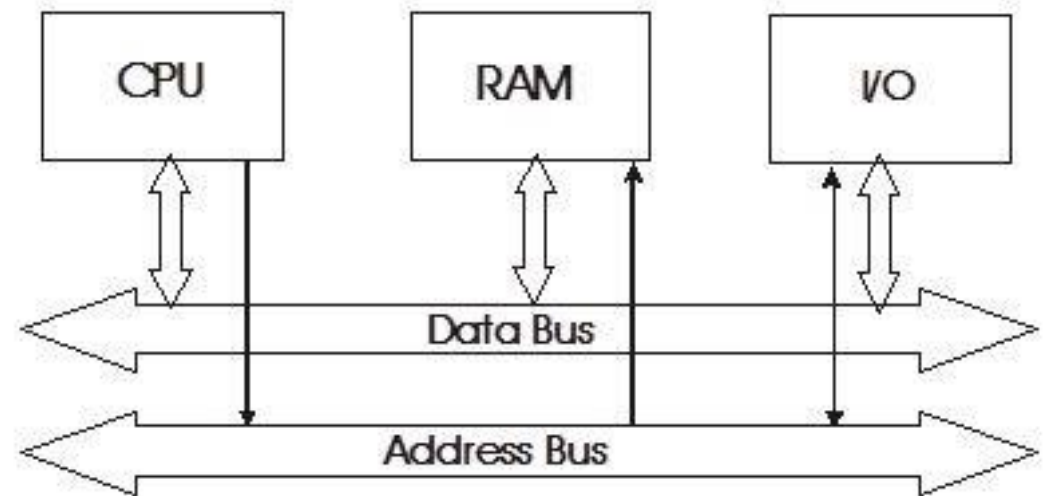
I/O Unit

Q.14

Describe the function of the I/O Unit of the computer system.

I/O Unit

- ✓ Input/output devices
- ✓ keyboard, mouse, scanner, monitor, printer, microphone, video camera, etc.
- ✓ Different functions
- ✓ Different data transfer rate
- ✓ Support different formats
- ✓ Impossible to connect directly to system bus
- ✓ Difficult for CPU to control directly.
- ✓ **Solution** Special hardware component *I/O unit*
- ✓ I/O unit control different I/O devices
- ✓ Only I/O unit connected to Bus
- ✓ CPU and all other devices are connected to I/O unit
- ✓ Works as an interface between CPU & I/O devices
- ✓ Performs all I/O operations or tasks without direct involvement of CPU
- ✓ Responsible to manage speed difference between processor & I/O devices



Ways of transferring data from peripheral devices into computer

1) Interrupts

- Signal sent by CPU I/O device (or sent by I/O device CPU)
- CPU issues command to I/O device for I/O operation
 - When device gets ready generates an interrupt signal to processor
 - CPU on receiving interrupt signal suspends all other processing & performs I/O operation
- + Requires no additional hardware
- + Easy to implement
- Processor suspends all other processing
 - Reduces overall performance of computer system

Ways of transferring data from peripheral devices into computer

2) DMA

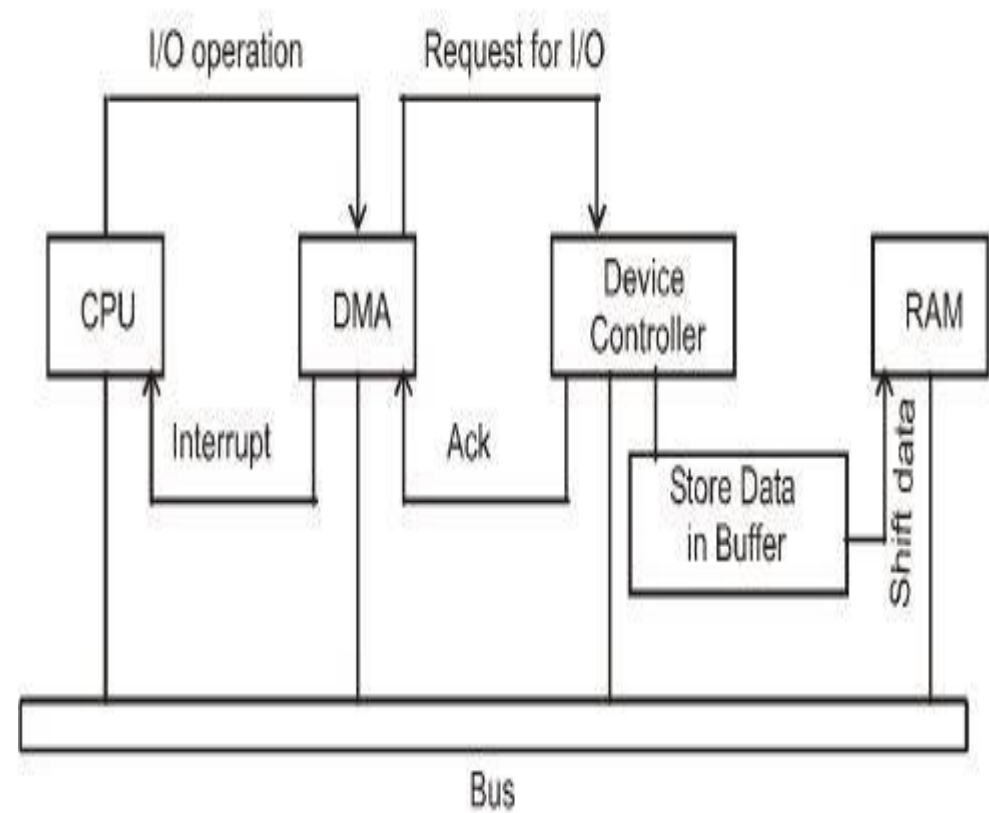
- DMA Direct Memory Access
- **DMA controller** special hardware component
- Data transfer takes place without involvement of CPU
- CPU sends request to DMA controller to perform I/O operation

- CPU continues to do other tasks
- When data transfer completed
- DMA controller sends on interrupt signal to CPU
- CPU involved only at beginning & end of data transfer

+Processor does not have to wait for completion of I/O operation

+Performance increased

-More complex & more hardware needed to implement

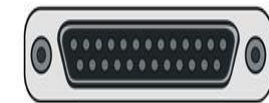
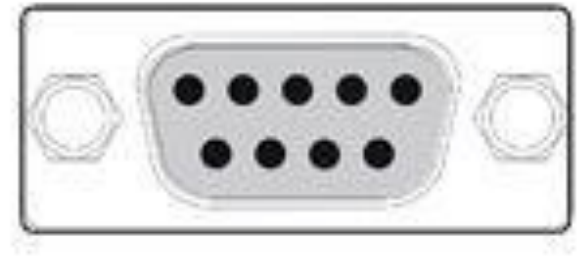


Port Q.22 What is a port? Explain different types of ports.

- Peripheral devices connected to system unit through a special device **Port**
- Port interface or connecting point
- Provide a standard way of communication between computer and its peripheral devices
 - Example: keyboard, mouse, printer, monitor, & modem connected with computer by plugging their connectors into ports
- Computer or mobile device different types of ports
 - Example: desktop personal computer different ports on its front and back

i) Serial Port

- Transmits one bit at a time
- communication (COM) port
- Slow data transmission
 - Data transmission rate 115 Kbps or more
- Serial port mouse, keyboard, and modem are connected
 - do not require fast data transmission
- Not used today



ii) Parallel Port

- Transmits many bits at a time
- Faster data transmission than serial port
 - Data transmission rate up to 12 Mbps or more
- Parallel port printer and scanner connected
- Line Printer Ports (LPT)
- Not used in modern computers
- Parallel and serial ports replaced with USB ports

CPU Registers

iii) USB Port

- USB Universal Serial Bus
- Most popular standard port used in PCs
- Single connector allows up to 127 peripheral devices to be connected
- Examples: keyboard, mouse, printer, scanner, digital camera, speakers, card reader, smartphone, USB flash drives, & external hard disk
- USB ports in personal computers typically 6 to 8 either on front or back
- Also be used as a power supply for different devices
 - such as smartphones & tablets



CPU Registers

CPU Registers

What are CPU registers? Describe the special-purpose registers.

Explain different general-purpose registers (GPR).

Explain the purposes of different Segment / Address registers.

CPU Registers

- **Registers** CPU contains small, high-speed storage locations (areas)
- Used during program execution
- Temporarily hold instructions, data, or intermediate results of calculations
- Different types of registers used for a specific purpose
- Size of registers 1, 2, 4, or 8 bytes
- Large register increases performance of CPU
 - Example: computer having 32-bit (4-bytes) registers
CPU can process four bytes of data at a time

• **Types of Registers**

- Special-purpose registers
- General-purpose registers
- Segment registers

Special Purpose Registers

- Perform specific functions (special functions) during program execution

CPU Registers

1. Program Counter (PC)

- Controls sequence in which instructions are fetched from memory
- Holds (stores) the address of next instruction to be fetched for execution
- When instruction fetched value of PC incremented
 - register always points or holds address of next instruction

• Instruction Pointer Register

2. Instruction Register (IR)

- Stores recently fetched instruction from memory
 - Control unit takes this instruction decodes, and executes it
- Special Purpose Registers

3. Memory Address Register (MAR)

- Holds address of memory where CPU wants to read/write
- When CPU wants to read/write places address of required memory location in the MAR

4. Memory Buffer Register (MBR)

- Store data coming from memory & going to memory

5. Stack Pointer

- Stack set of memory blocks (data stored and retrieved in order)
- Order Last In, First Out (LIFO)
- Stack pointer register used to manage stacks in memory
 - For example: undo & redo operations in different application software

CPU Registers

General-Purpose Registers

- Store data (operands), addresses of data and intermediate results of operations
- Used for arithmetic & logical operations, data movement
- Part of ALU

1. Accumulator Register (AX)

- Used for arithmetic and other data operations

2. Base Register (BX)

- Used during arithmetic, logic, and data movement operations
- Stores memory addresses of data

3. Counter Register (CX)

- Used for counting purposes
- Acts as a counter for looping

4. Data Register (DX)

- Used with Accumulator register for division & multiplication operations •
- Also used in input/output operations

General-Purpose Registers

CPU Registers

- Divided into a higher & lower byte
 - ✓ Lower byte registers AL, BL, CL, and DL
 - ✓ Higher byte registers AH, BH, CH, and DH
 - ✓ A Accumulator
 - ✓ B Base
 - ✓ C Count
 - ✓ D Data
 - ✓ H Higher
 - ✓ L Lower

31	16 15	8 7	0	16-bit	32-bit
	AH	AL		AX	EAX
	BH	BL		BX	EBX
	CH	CL		CX	ECX
	DH	DL		DX	EDX
	BP				EBP
	SI				ESI
	DI				EDI
	SP				ESP

- Size lower byte & higher byte 1 byte (8 bits)
- Size of AX, BX, CX, and DX 2 bytes (16 bits)
- Size of EAX, EBX, ECX, and EDX 4 bytes (32 bits)
- Modern processors use 4 bytes registers EAX, EBX, ECX, and EDX

CPU Registers

Segment Registers

- **Segment** A block of memory
- **Segment Registers** store addresses of memory blocks that are being currently used by CPU, **Address registers**
- Four segment registers CS, DS, ES, & SS
- Size 2-bytes
- Used with IP register or with two index registers DI & SI to address various areas or blocks of memory
- **CS (Code Segment)**
 - Holds base location of all executable instructions (code) of program
 - Used along with IP register to fetch instruction
- **DS (Data Segment)**
 - Used to store base location of memory variables
 - Used along with DI or SI register to refer to data
- **ES (Extra Segment)**
 - Used as an additional base location for memory variables
- **SS (Stack Segment)**
 - Used to store base location of current program stack

Every address has two parts, a **SEGMENT** and an **OFFSET**
(Segment:Offset)

CPU Registers

Instruction Pointer

- 16-bit
- Always points to the next instruction to be executed within the currently executing code segment.
- So, this register contains the 16-bit offset address pointing to the next instruction code within the 64Kb of the code segment area.
- Its content is automatically incremented as the execution of the next instruction takes place.

Stack Pointer (SP) and Base Pointer (BP)

- SP and BP are used to access data in the stack segment.
- SP is used as an offset from the current SS during execution of instructions that involve the stack segment in the external memory.
- SP contents are automatically updated (incremented/ decremented) due to execution of a POP or PUSH instruction.

BP contains an offset address in the current SS, which is used by instructions utilizing the based addressing

Source Index (SI) and Destination Index (DI)

- Used in indexed addressing.
- Instructions that process data strings use the SI and DI registers together with DS and ES respectively in order to distinguish between the source and destination addresses.

CS:IP

SS:SP SS:BP

DS:BX DS:SI

DS:DI (for other than string operations)

ES:DI (for string operations)